```
»     end
»   end
```

Path configuration is comprised of two parts: `settings` and `rules` keys. The `settings` key isn't used directly by the framework. It's provided for developers to define and manage their own app-specific configuration. Anything you want to configure on the server but apply in the app belongs here.

This chapter focuses on the `rules` key—this is where we map URL paths to navigation options. Think of these rules as a way to abstract behavior from hardcoded URLs.

A completely empty path configuration for iOS looks like the following:

ch03_03/rails/app/controllers/configurations_controller.rb

```
    class ConfigurationsController < ApplicationController
      def ios_v1
        render json: {
»         settings: {},
»         rules: [
»         ]
        }
      end
    end
```

This configures no custom settings and no rules. Even though we won't use any custom iOS settings in this book, we still need to include the empty object ("hash," in Ruby terms) for Hotwire Native to work correctly.

The `rules` key holds an array of objects, each with two keys: `patterns` and `properties`. The `patterns` key matches URL paths via a regular expression, and the `properties` key applies different behaviors to the route.

Hotwire Native automatically handles a few different `properties` out of the box. For now, we'll focus on the *context* property, which tells the framework